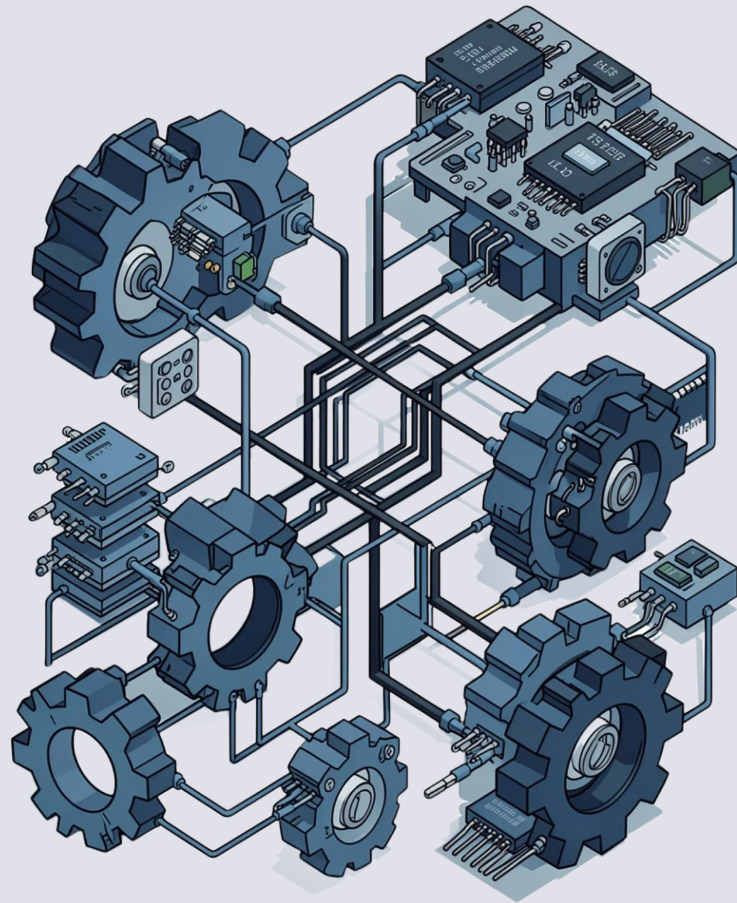




Structured Outputs, JSON & Function Calling

Unit 2 — Part 12: How LLMs Communicate with Software

AGENT ENGINEERING · INTERMEDIATE TRACK

The Problem with Normal LLM Output

What the user asks

"Find the highest-paid employee."

The LLM responds with a natural language explanation — plus an embedded SQL query buried in prose.

Why this breaks pipelines

- ✗ Python can't reliably parse free-form text
- ✗ No deterministic structure to extract
- ✗ Application can't execute without fragile manual parsing
- ✗ Different runs produce differently formatted output



What We Need Instead

Structured Format

Machine-readable output the application can parse deterministically — every single time.

Clear Action + Params

A defined action name paired with typed, validated parameters — no ambiguity.

Python-Ready

Output that maps directly to Python dicts, function args, or API payloads without any regex hacks.

Deterministic Parsing

Same input shape, same output shape — zero guesswork for downstream code.

Structured Output Example

✗ Before — Natural Language

"I will query the database to find
the employee with the highest salary.
Here is the SQL I would use:
SELECT name, salary FROM employees
ORDER BY salary DESC LIMIT 1"

Buried in prose. Unparseable by code without fragile string extraction.

✓ After — Structured JSON

```
{  
  "action": "execute_sql",  
  "query": "SELECT name, salary  
           FROM employees  
           ORDER BY salary DESC  
           LIMIT 1"  
}
```

Direct mapping to a Python dict. One `json.loads()` call and you're ready to execute.

Why JSON?



Machine-Readable

Parse with a single call — `json.loads()` in Python, `JSON.parse()` in JS. No regex. No fragile string splitting.



Structured Key-Value Pairs

Every field is explicitly named. Access `response["action"]` directly — no positional parsing.



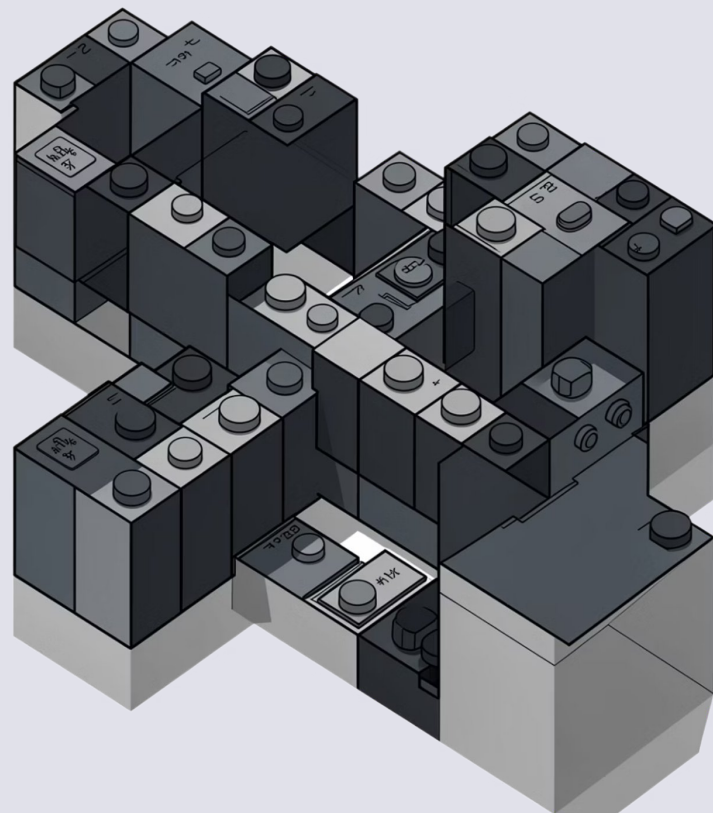
Language-Independent

Works identically in Python, JavaScript, Go, Ruby, or any language with an HTTP client.

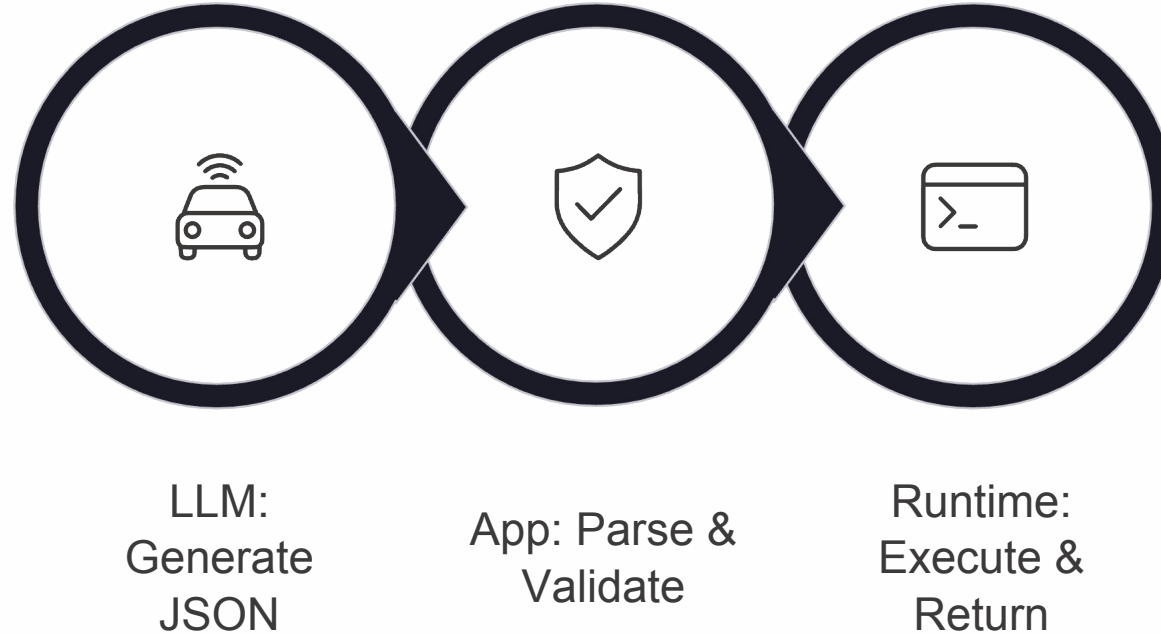


API-Compatible

The universal standard for web services. Every REST API speaks JSON — your agent fits right in.



The Core Pipeline



LLM decides *what* should happen

The model interprets intent and produces a structured request — never touching your runtime directly.

Application controls *what actually happens*

Your code validates, gates, and executes. This separation is the cornerstone of agent security.

LLM vs Application: The Critical Distinction

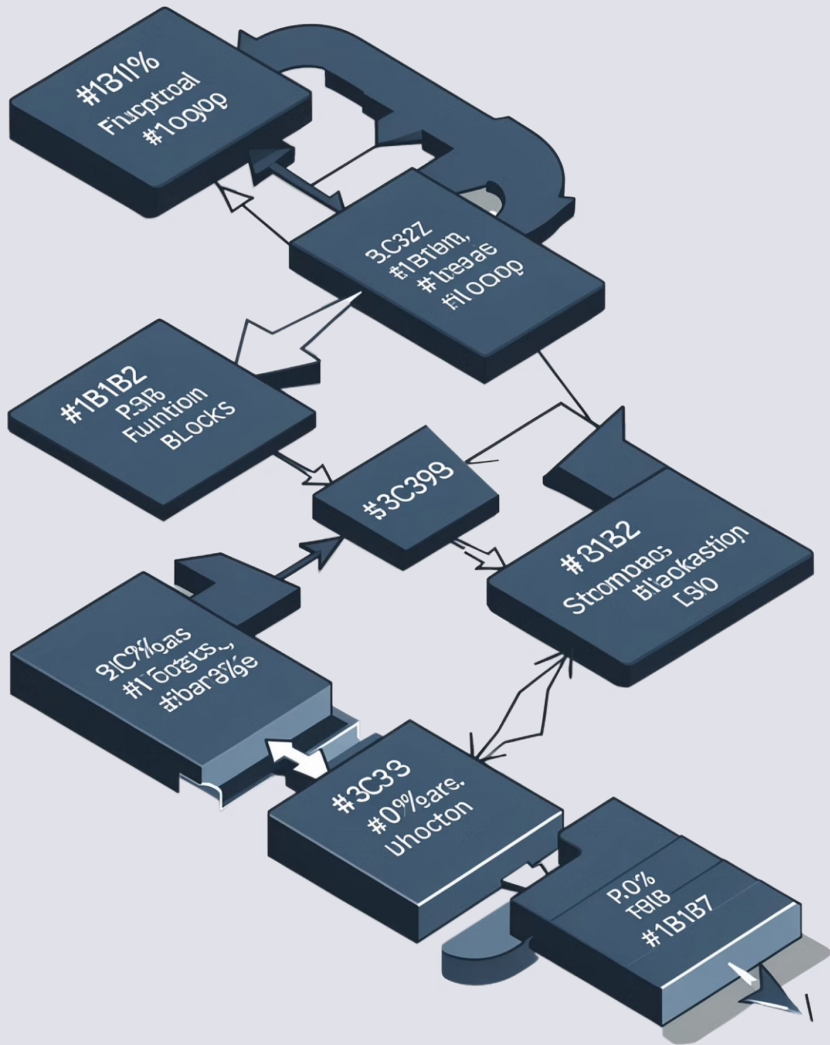
LLM Role

- "I need to call the calculator"
- Generates a structured request object
- Receives the result and forms a final response
- **Never directly executes your code**

Application Role

- Receives the LLM's structured request
- Validates: Is this call permitted?
- Executes the tool safely
- Returns result back to the LLM

 The application is the gatekeeper — always.



Function Calling Basics

01

Expose Definitions

Application provides the LLM with a schema of available functions — names, parameters, and descriptions.

02

LLM Generates Request

Model interprets user intent and emits a structured JSON call targeting one of the defined functions.

03

Validate & Execute

Application validates the request, checks permissions, then executes the real function in your runtime.

04

Return Result

Execution result is passed back to the LLM, which uses it to compose the final user-facing response.

Function Definition Example

Python Function + Schema

```
def get_weather(city: str) -> dict:
    """Get current weather for a city."""
    return weather_api.fetch(city)

# Schema exposed to the LLM:
{
  "name": "get_weather",
  "description": "Get current weather for a city",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string",
        "description": "City name"
      }
    },
    "required": ["city"]
  }
}
```

What the LLM sees

- **Name:** get_weather
- **Input:** city (string, required)
- **Description:** Natural language hint for when to use it

The LLM uses this schema to generate *only valid, well-typed* function calls — it cannot invent parameters that aren't declared.

Function Calling in Action

1

User Input

"What's the weather in Pune?"

2

LLM Generates

```
{"name": "get_weather",  
  "arguments": {  
    "city": "Pune"  
  }}  
}
```

3

App Executes

```
result = get_weather(  
  "Pune"  
)
```

4

Result Returned

```
{"temperature": 28,  
  "condition":  
    "Rain"}
```

The LLM never touches the weather API — it only **requests**. Your application **executes**.

Function Calling ≠ Function Execution

What "Function Calling" Actually Means

```
# LLM produces this JSON — nothing more:
```

```
{  
  "name": "get_weather",  
  "arguments": {"city": "Pune"}  
}
```

```
# Your app reads it and decides:
```

```
if is_allowed("get_weather"):  
    result = get_weather("Pune")  
else:  
    result = {"error": "Not permitted"}
```

The Golden Rule

The LLM says "**Call get_weather with Pune.**" Your Python app actually runs `get_weather("Pune")`.

The LLM doesn't magically execute your code. **Your application is the gatekeeper** — it validates every request before a single line of your logic runs.

📌 Security principle: **LLM should request. Application should decide.**